D.K.T.E. Society's Textile and Engineering Institute, Ichalkaranji.
(An Empowered Autonomous Institute, Affiliated to Shivaji University, Kolhapur)
Department of Computer Science & Engineering (Artificial Intelligence and Machine Learning)
2025-26

A Project Synopsis
On

AgriSense: A Deep Learning Based Web Platform for Crop Disease Detection, Weather-Driven Risk Advisory and Market Linkage

SUBMITTED BY

Mr. Jatin Sanjaykumar Dayama 23UAM025
Mr. Bapurao Anil Kundale 23UAM066

UNDER THE GUIDANCE OF

[Project Guide] [I/C Director] HOD (CSE(AI & ML))

1. Title of Project

AgriSense: A Deep Learning Based Web Platform for Crop Disease Detection, Weather-Driven Risk Advisory and Market Linkage

2. Introduction & Background

Agriculture supports the livelihood of a very large share of the rural population in India, and crop loss caused by plant disease remains one of the most damaging and most preventable sources of that loss. A disease that is identified in its early stage can usually be controlled with a low-cost intervention, whereas the same disease identified two weeks later often means a significant reduction in yield.

The practical difficulty is not that treatments are unknown; it is the delay between the moment a farmer notices something wrong on a leaf and the moment a reliable diagnosis reaches that farmer. In many villages the nearest agricultural officer serves a large number of holdings, visits are infrequent, and the advice a farmer receives in the meantime often comes from a neighbour or from a shop that sells the product it recommends.

AgriSense is a web-based platform that reduces this delay. A farmer registers once, records the village location and preferred language, and then has access to five capabilities from a single dashboard:

- Photographing a diseased leaf to receive a diagnosis produced by a convolutional neural network trained on a labelled plant disease dataset.
- Reading the symptoms, causes, preventive practices and recommended treatment for the identified disease.
- Viewing current and forecast weather for the registered location.
- Receiving an automatic alert when weather conditions become favourable for a known crop disease, before any symptom is visible.
- Browsing a directory of registered buyers and retailers and contacting them directly.

The project therefore brings together deep learning, retrieval-augmented generation, external meteorological services, relational data management and full-stack web development into one working system aimed at a genuine field problem.

Motivation

The motivation for this work rests on three observations. First, image-based plant disease classification has become technically accessible: publicly available labelled datasets and pretrained convolutional architectures make it possible to build an accurate classifier without specialised hardware or a proprietary dataset. Second, the value of a classifier depends almost entirely on how it is delivered — a model behind a simple web page, returning an answer in the farmer's language along with a clear treatment recommendation, becomes a genuinely usable tool, and this project treats deployment and usability as part of the engineering problem rather than an afterthought. Third, prevention is more valuable than detection: warning a farmer before an outbreak begins, using freely available weather

data and documented disease-favourability conditions, can prevent crop loss rather than merely limiting it.

3. Literature Review / Existing Systems

A review of existing academic work and deployed applications in this space reveals a consistent pattern: individual capabilities are well studied in isolation, but they are rarely integrated into a single farmer-facing system.

3.1 Image-Based Plant Disease Classification

Convolutional neural networks, particularly through transfer learning on architectures such as MobileNet, ResNet and VGG pretrained on ImageNet, have been shown to achieve high accuracy on public plant disease datasets such as PlantVillage [1], [2]. These studies establish that the technical barrier to building an accurate classifier has largely been removed, but most stop at reporting laboratory-dataset accuracy without addressing deployment, explanation of results, or generalisation to field-captured images.

3.2 Weather-Driven Disease Advisory

Agrometeorological research has long established quantitative relationships between temperature, humidity and the onset of major fungal and bacterial crop diseases, such as late blight in potato and tomato [3]. This knowledge is well documented in agronomic literature but is rarely delivered to individual farmers in a personalised, location-aware, preventive form.

3.3 Retrieval-Augmented Generation for Domain Question Answering

Retrieval-augmented generation (RAG) has emerged as an effective approach for grounding large language model responses in a trusted corpus rather than relying on unconstrained generation, reducing hallucination in domain-specific question answering [4]. Its application to agricultural extension advisory, where answers must be traceable to a verifiable source, remains limited in existing farmer-facing tools.

3.4 Existing Mobile and Web Applications

Several deployed applications address parts of this problem individually. Some mobile applications classify leaf images and return a disease name; separate applications report weather; separate portals list agricultural market or mandi information. Existing image classification applications generally return a label with no actionable follow-up, offer no explanation of causes or preventive practice, provide no way for the farmer to ask a follow-up question, and make no use of the environmental conditions that caused the outbreak in the first place. They treat diagnosis as an isolated event rather than as one step in a season-long decision process, and none of these tools shares context — such as the farmer's location, crop or detection history — with the others.

3.5 Research Gap

The literature and existing systems, taken together, show that accurate disease classification, weather-based risk modelling, grounded conversational advisory and market connectivity all exist as separate, well-studied capabilities, but no widely available system unifies them around a single farmer profile so that each capability can reinforce the others. This is the gap that AgriSense is designed to address.

4. Problem Statement

Farmers who cultivate small and medium holdings face a recurring diagnostic gap. When a crop begins to show disease symptoms, the farmer must decide quickly what the disease is and how to respond, but the information available at that moment is often unreliable. The problem has four distinct parts.

Delayed and inconsistent diagnosis. Expert advisory services are limited relative to the number of holdings they serve. A farmer who cannot reach an expert within a few days either applies a treatment on the basis of guesswork or waits until the infection has spread. Incorrect identification leads to the wrong chemical, wasted expenditure, unnecessary chemical load on the soil and continued crop loss.

Reactive rather than preventive action. Most disease response begins only after symptoms appear. Many important fungal and bacterial diseases, however, are strongly driven by weather, following well-documented temperature and humidity conditions that occur before any visible symptom. This predictive information is published in agricultural literature but is not delivered to the individual farmer in a usable form.

Fragmented digital tools. Several isolated solutions exist — some mobile applications classify leaf images, separate applications report weather, and separate portals list market information. A farmer with limited digital experience is unlikely to install, learn and maintain four different applications, and none of these tools shares context with the others.

Limited market reach. Produce frequently moves through several intermediaries between the field and the final buyer, which reduces the share of the final price that reaches the farmer.

Why Existing Solutions Are Insufficient

Existing image classification applications generally return a disease name with no actionable follow-up, offer no explanation of causes or preventive practice, provide no way for the farmer to ask a follow-up question, and make no use of the environmental conditions that caused the outbreak in the first place. They treat diagnosis as an isolated event rather than as one step in a season-long decision process.

How This Project Addresses the Problem

AgriSense integrates diagnosis, explanation, conversation, prevention and market access into a single farmer-facing web application built on one user profile and one location record. The deep learning model answers the question of what the disease is. The curated knowledge base answers what should be done about it. The retrieval-augmented assistant answers the follow-up questions that the fixed knowledge base cannot anticipate. The weather-driven risk engine warns the farmer before the next outbreak begins. The buyer directory addresses the point at which the harvested crop is sold. Because

these modules share the same farmer profile and the same location, each one improves the usefulness of the others.

5. Objectives & Scope

5.1 Objectives

1. To design and develop a farmer-centric web application providing secure registration, authentication, profile management, location storage and language preference for users with limited digital experience.
2. To build and train a convolutional neural network using transfer learning for the classification of crop leaf diseases from uploaded images, and to evaluate it against a baseline model trained from scratch.
3. To construct a curated agricultural knowledge base that maps every disease class predicted by the model to its symptoms, probable causes, preventive practices and recommended treatment, and to present this information in the farmer's preferred language.
4. To integrate current and forecast meteorological data for the farmer's registered location through public weather APIs and display the agriculturally relevant parameters on the dashboard.
5. To implement a rule-based agro-meteorological risk engine that evaluates observed and forecast weather against documented disease-favourability criteria and generates preventive alerts before symptoms appear.
6. To develop a retrieval-augmented question-answering assistant that grounds its responses in a trusted agricultural corpus, so that farmers receive answers supported by source documents rather than unverified generated text.
7. To provide a searchable directory of registered buyers and retailers that allows farmers to identify and contact purchasers of their crop directly.
8. To evaluate the complete system in terms of classification accuracy, per-class precision, recall and F1 score, confusion matrix analysis, and end-to-end response time of the deployed inference pipeline.

5.2 Scope

The current version deliberately excludes several capabilities so that the core system can be completed and evaluated properly within the project timeline. In scope for this project are: farmer and retailer registration and authentication; CNN-based leaf disease classification for a defined set of region-relevant crops; a curated multilingual disease knowledge base; a retrieval-augmented conversational assistant grounded in agricultural extension documents; a rule-based weather-driven preventive risk engine; and a browsable buyer/retailer directory. Out of scope for the current version are direct transactions and payments between farmers and buyers, logistics coordination, market price forecasting, a native mobile application, and sensor-based (IoT) data collection — these are identified as future work in Section 10 (Future Scope) of the full project report.

6. Proposed System & Methodology

The system is built as a three-tier web application. A browser-based client communicates over REST with an application server, which in turn coordinates a relational database, a locally hosted trained model, a vector store and two external services.

6.1 Methodology — Stage-Wise Plan

Stage 1 — Data acquisition and model development. A publicly available labelled plant disease image dataset is obtained and reduced to a set of crops relevant to the region of deployment. Images are split into training, validation and test partitions in a stratified manner before any augmentation is applied, so that no augmented variant of a training image can appear in the test set. Images are resized to a fixed input resolution and normalised; augmentation covering rotation, translation, zoom, horizontal flip and brightness variation is applied to the training partition only.

Stage 2 — Transfer learning. A MobileNetV3 backbone pretrained on ImageNet is used as a feature extractor. The original classification head is discarded and replaced with a global average pooling layer, a dropout layer and a softmax output layer sized to the number of disease classes. Training proceeds in two phases: first with the backbone frozen and only the new head trained at a moderate learning rate, then with the upper convolutional blocks unfrozen and the whole network fine-tuned at a much lower learning rate. Early stopping, learning-rate reduction on plateau and best-checkpoint restoration are used throughout.

Stage 3 — Model evaluation and export. The trained model is evaluated on the held-out test partition and additionally on a small set of photographs captured under realistic field conditions, in order to report generalisation honestly rather than only on laboratory-style images. The final weights are exported and loaded once into the application server at startup, so that inference does not require reloading the model on each request.

Stage 4 — Knowledge base construction. Two related resources are prepared from the same source material: a structured table of disease records (one per model class) holding symptoms, causes, prevention and treatment in each supported language, and a document collection derived from agricultural extension publications and university advisories, segmented into passages, converted into dense vector embeddings and stored in a vector database with source metadata.

Stage 5 — Retrieval-augmented generation. When a farmer asks a question, it is embedded using the same embedding model, the most semantically similar passages are retrieved from the vector store, and those passages are supplied as grounding context to a large language model together with the question, the farmer's crop and the most recent detection result. The model is instructed to answer only from the provided context, to state clearly when the context is insufficient, and to respond in the farmer's preferred language, with source references returned alongside the answer.

Stage 6 — Risk engine. A scheduled background job runs once daily. For each distinct registered location it retrieves hourly forecast data, evaluates a set of stored disease-favourability rules against that data, and where a rule is satisfied it creates a risk record and a notification for every farmer at that location who grows the affected crop.

Stage 7 — Application layer. All modules are exposed through a single authenticated web interface. Each farmer sees only their own detection history, alerts and profile; retailers register under a separate role and appear in the public buyer directory.

6.2 System Architecture

The architecture follows a layered design: a React-based client communicates over HTTPS/REST (JWT-authenticated) with a FastAPI application server, which coordinates seven internal services (authentication, profile, weather, detection, RAG, risk and market) against four backing resources.

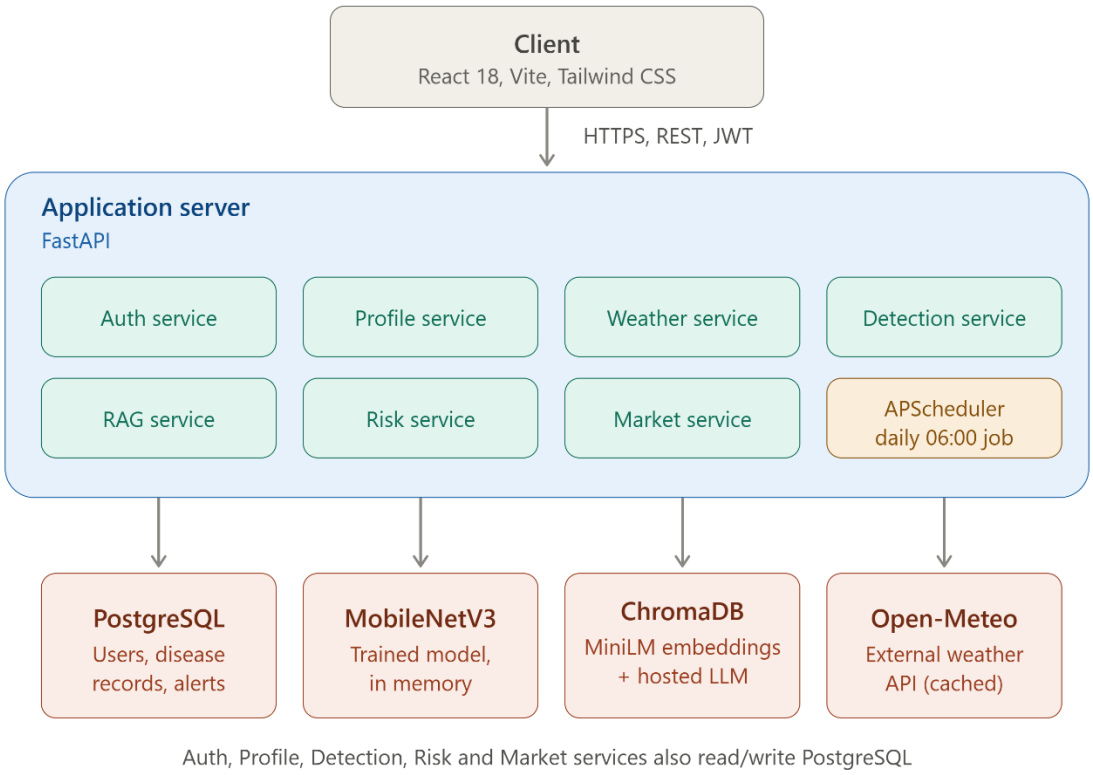


Fig.1 System Architecture

Layer / Resource	Role in the System
Client	React 18 + Vite + Tailwind CSS browser interface; the farmer's single dashboard.
Application server	FastAPI server hosting Auth, Profile, Weather, Detection, RAG, Risk and Market services.
PostgreSQL	Stores users, disease records, detection history and alerts (strongly relational data).
MobileNetV3 model	Trained classifier (.keras), loaded once in memory for the lifetime of the process.
ChromaDB + MiniLM + LLM	Persistent vector store, sentence-transformer embeddings and hosted LLM for grounded RAG answers.
Open-Meteo API	External current/forecast weather source; responses cached per location per hour.
APScheduler	Runs the daily risk-evaluation job independent of user requests.

Table 1. System architecture — layers, services and backing resources.

6.3 Primary User Flow

Registration and login (name, phone, password, village, language) issues a JWT session and leads to the farmer dashboard, from which five modules are reachable: the Weather Panel, Disease Detection, the RAG Assistant, Risk Alerts and the Market Directory.

6.4 Disease Detection Sub-Flow

A farmer uploads a leaf image, which is validated (type, size, resolution) and preprocessed (resized to 224×224, normalised) before CNN inference produces a softmax vector over N classes. The top-1 class, its confidence score and the top-3 alternatives are computed, the corresponding disease record is looked up in PostgreSQL, and symptoms, causes, prevention and treatment are fetched in the farmer's preferred language. The result is stored in the detection history and rendered with an option to ask a follow-up question, which pre-seeds the RAG assistant with the detected disease.

6.5 Preventive Alert Sub-Flow

A daily scheduled job (APScheduler, 06:00) collects distinct farmer locations from the database and, for each, fetches a 48-hour hourly forecast (temperature, relative humidity, rainfall). Stored disease-favourability rules are evaluated against this data — for example, hours where temperature lies between 10–24°C together with relative humidity of 90% or more. Where a rule is satisfied, a risk-alert record is created together with a notification for every farmer at that location growing the affected crop; the farmer sees the alert on next login, linked to prevention guidance for that disease.

7. Tools & Technical Stack

The technology choices favour free and open-source components wherever possible, with a Python-centric backend chosen specifically because the trained model is a Python object — keeping inference in-process removes the need for a second microservice, a message queue or a cross-language bridge.

Layer	Technology
Frontend	React 18, Vite, Tailwind CSS, Axios, React Router
Backend	Python 3.11, FastAPI, Uvicorn, Pydantic
Authentication	JWT, passlib with bcrypt password hashing
Database	PostgreSQL 16, SQLAlchemy ORM, Alembic migrations
Deep learning	TensorFlow, Keras, MobileNetV3, trained on Google Colab GPU
Image handling	Pillow, NumPy
Vector store	ChromaDB (persistent local mode)
Embeddings	sentence-transformers, all-MiniLM-L6-v2
Language model	Hosted LLM (free tier API), with local Ollama as offline fallback
Weather data	Open-Meteo API
Scheduling	APScheduler
Version control	Git, GitHub
Deployment	Backend on Render, frontend on Vercel, database on Neon

Table 2. Technologies used across the AgriSense stack.

Expected Outcome

On completion, the project is expected to deliver a trained crop disease classifier evaluated with per-class precision, recall, F1 score and a full confusion matrix, quantified against a from-scratch baseline; a functioning end-to-end web application covering all five farmer-facing modules; inference response times suitable for interactive use on a standard server without a GPU; and a complete relational schema, API documentation and a reproducible training notebook.

8. References

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," Frontiers in Plant Science, vol. 7, art. 1419, Sept. 2016.

[2] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, "Searching for MobileNetV3," in Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), Seoul, South Korea, 2019, pp. 1314–1324.

[3] W. E. Fry, "Phytophthora infestans: The plant (and R gene) destroyer," Molecular Plant Pathology, vol. 9, no. 3, pp. 385–402, May 2008.

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459–9474.

[5] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), Miami, FL, USA, 2009, pp. 248–255.

[6] Open-Meteo, "Open-Meteo Weather API documentation," Open-Meteo.com, 2024. [Online]. Available: https://open-meteo.com/en/docs. [Accessed: Sept. 1, 2026].

[7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), Hong Kong, China, 2019, pp. 3982–3992.

[8] FastAPI, "FastAPI documentation," FastAPI.tiangolo.com, 2024. [Online]. Available: https://fastapi.tiangolo.com. [Accessed: Sept. 1, 2026].